MAPÚA MALAYAN DIGITAL COLLEGE
of Mapúa Malayan Colleges Laguna
In collaboration with
Arizona State University®

MO-IT153 Information Assurance and Security 2

Security Controls Audit Worksheet

System Overview & Scope	
Project/System Name	MotorPH Payroll Management System (Advanced OOP Version)
Team Members and Roles	ABIGAIL AGNES - Security Lead / Backend Integration
	Keon Clark de Leon - Architecture & Data Layer Specialist
	Ron Carlo Dioso - Security Auditor / Vulnerability Assessment
	Mikee Miguel - Quality Assurance & Compliance Documentation
System Description	A Java-based enterprise application designed to automate employee compensation processing, maintain organizational personnel records, and generate payroll financial summaries for MotorPH staff, payroll administrators, and HR managers.
Architecture Summary (components, tech stack)	**Frontend / Interface:** Web-based interface transmitting HTTP/HTTPS requests to backend routing endpoints. **Backend Logic:** Java OOP application layer housing business logic, gross/net compensation formula calculators, and JDBC database access objects. **Database Layer:** Relational MySQL database holding employee profiles, authentication credentials, and historical payroll ledgers. **Runtime Configuration:** Local system properties and configuration files defining database connection URLs and runtime flags.
Audit Scope (what's in/out)	***In-Scope:*** Application-layer authentication routines, database querying mechanisms (JDBC search/lookup builders), credential storage schemas, server-side authorization checks on compensation endpoints, and concurrency handling under simulated load. ***Out-of-Scope:*** Third-party cloud services (AWS SQS/EC2 infrastructure), underlying physical server hardware, operating system kernel tuning, and client-side web browser vulnerabilities.
Tools & Methods	
Chosen Automated Tools (with reasons):	**OWASP ZAP / Burp Suite Community:** Chosen to intercept web traffic, inject SQL payloads, fuzz search parameters, and analyze server response codes. **GitGuardian / TruffleHog CLI:** Selected to scan local repositories and working directories for hardcoded API keys, database connection strings, and administrative secrets. **Apache JMeter:** Chosen to generate concurrent HTTP request bursts simulating peak payday traffic and evaluate thread pool behavior.
Chosen Manual Techniques (with reasons):	**Static Code Review:** Manual line-by-line inspection of JDBC query construction methods, variable access modifiers, and session handling logic. **HTTP Request Parameter Manipulation:** Manual tampering of query strings and URL path parameters (`/api/employee/{id}/payroll`) using Postman and browser Developer Tools to test object-level boundaries. **Direct Database Inspection:** Running SQL queries directly against MySQL database tables (`users, employees, payroll_records`) via CLI to audit raw data formatting and hash values.
Justification for each tool/method per system area (frontend, backend, etc):	**Presentation / Gateway Layer:** Postman and JMeter provide direct HTTP manipulation and load stress capabilities without requiring full browser instrumentation. **Application Logic Layer:** Static code review combined with OWASP ZAP fuzzing isolates dynamic string concatenation and missing authorization checks at the method level. **Data & Configuration Layer:** GitGuardian automated scanning and MySQL CLI inspection directly verify whether database credentials and user passwords are exposed in cleartext.
Review of Initial Security Control Proposals (IAS 1)	
Summary of Key Threats/Risks Identified:	**RISK-001 (Priority 3):** Unauthorized Salary Peeking via client-side access assumptions and horizontal privilege escalation. **RISK-002 (Priority 1):** SQL Injection via unsanitized employee search inputs using dynamic string concatenation. **RISK-003 (Priority 3):** Payroll Logic Tampering through unsecured backend calculation variables and internal execution paths. **RISK-004 (Priority 2):** Credential Exposure due to hardcoded administrative database connection strings and plain-text passwords in internal repositories. **RISK-005 (Priority 2):** Payday System Crash caused by synchronous processing loops and thread starvation under peak concurrency.
Controls Proposed in IAS 1:	**Control 1:** Server-side session validation and object-level authorization via signed JWTs. **Control 2:** Pre-compiled `PreparedStatement` parameters across all JDBC database interactions. **Control 3:** Private class encapsulation and SHA-256 cryptographic check hashes for payroll logs. **Control 4:** Externalized environment variables (`.env`) and salted `bcrypt` password hashing. **Control 5:** Token Bucket rate limiting combined with asynchronous background worker queues.

	Assumptions from IAS 1:	Assumed the corporate internal network perimeter provides sufficient isolation against unauthorized external access. Assumed end users interact solely through the rendered client UI without intercepting or forging raw HTTP requests. Assumed external cloud services (AWS SQS/EC2) could be integrated into testing without billing constraints or local environment friction.
	Audit Goals	
	Specific Controls or Areas to Validate:	Input validation and query execution paths in employee lookup and search modules. Credential security in local source code repositories and storage format in database tables. Session validation and access control enforcement on sensitive compensation endpoints. Application throughput and thread pool resilience under simulated concurrent traffic.
	High-Risk Areas for Focus:	**Priority 1 – Database Query Construction (RISK-002):** Dynamic SQL concatenation creates an immediate risk of full relational database exfiltration. **Priority 2 – Hardcoded Credentials & Plain-Text Passwords (RISK-004):** Exposed database credentials grant administrative root access across the application environment. **Priority 3 – Object-Level Access Control (RISK-001):** Missing backend identity verification permits employees to peek at executive compensation, violating Republic Act No. 10173 (Data Privacy Act of 2012).
	Audit Process & Evidence	
	Audit Area 1: Database Query Parameterization (RISK-002)	
	Control/Area Tested:	Employee Search & Retrieval JDBC Query Methods
	Automated Tool Used:	OWASP ZAP (Active Scanner / Fuzzing Module)
	Manual Method Used:	Manual SQL payload injection (`' OR '1'='1' --`) directly into search input fields via browser and Postman
	Evidence Collected (describe & attach files/screenshots):	SQL syntax error log leaks database structure (`SQLException: syntax error near ''1'='1'`); query returns full dump of all employee records instead of filtered result.
	Result (pass/fail/partial):	Fail
	Notes/Observations:	The application concatenates raw strings into SQL statements (`"SELECT * FROM employees WHERE lastName = '" + input + "'"`), confirming high vulnerability to injection.
	Audit Area 2: Credential & Secret Management (RISK-004)	
	Control/Area Tested:	Database Connection Handler & User Authentication Table
	Automated Tool Used:	GitGuardian CLI repository scan
	Manual Method Used:	Static inspection of `DatabaseConnection.java` source code and `SELECT username, password FROM users;` in MySQL CLI
	Evidence Collected (describe & attach files/screenshots):	GitGuardian scan detected hardcoded database password in git commit history; MySQL table displayed raw plain-text strings in the `password` column.
	Result (pass/fail/partial):	Fail
	Notes/Observations:	Administrative root credentials exist directly inside source repository files, and user passwords lack cryptographic salting or hashing.
	Audit Area 3: Access Control & Authorization (RISK-001)	
	Control/Area Tested:	Employee Payroll Record Endpoint (`/api/employee/{id}/payroll`)
	Automated Tool Used:	Burp Suite Community (HTTP History & Repeater)
	Manual Method Used:	Authenticated as standard Employee A (`id=1004`), intercepted GET request, modified parameter to Employee B (`id=1001`), and replayed request
	Evidence Collected (describe & attach files/screenshots):	Server responded with HTTP 200 OK and delivered the full salary breakdown of Employee B to Employee A's session.
	Result (pass/fail/partial):	Fail
	Notes/Observations:	Authorization relies on client-side interface element masking; no server-side identity-to-resource ownership validation exists on the endpoint.
	Audit Area 4: Request Concurrency & Availability (RISK-005)	
	Control/Area Tested:	Bulk Payroll Calculation Endpoint (`/api/payroll/calculate-all`)
	Automated Tool Used:	Apache JMeter (Thread Group: 100 concurrent virtual users over 5 seconds)

		Manual Method Used:	Manual browser refresh during active JMeter execution run				
		Evidence Collected (describe & attach files/screenshots):	Application response times degraded from 120ms to >15,000ms; server threw `OutOfMemoryError: Java heap space` and dropped incoming connections.				
		Result (pass/fail/partial):	Fail				
		Notes/Observations:	The endpoint runs intensive synchronous loops on the main servlet thread without rate limiting or background queue dispatching.				
	Analysis & Comparison						
	Summary Table		**Control**	**IAS 1 Expectation**	**Audit Result**	**Effective?**	**Change Needed?**
			Server-Side Session & Object-Level Authorization	Reject unauthorized ID queries with HTTP 403 Forbidden.	Server returned HTTP 200 OK with foreign employee salary data	No	Yes: Implement server-side JWT verification filter and enforce ownership checks before returning records.
			Parameterized Queries via `PreparedStatement`	Neutralize SQL meta-characters and treat inputs strictly as parameter literals.	Raw SQL concatenation caused dynamic query manipulation and full record dump.	No	Yes: Refactor all JDBC query builders to use pre-compiled `PreparedStatement` objects.
			Cryptographic Ledger Hashing & Encapsulation	Invalidate tampered payroll records via broken SHA-256 hash chains.	Records are stored without cryptographic checksums or validation hashes.	No	Yes: Maintain control concept, but lower priority relative to high-risk input injection and credential exposure.
			Externalized Config (`.env`) & Salted `bcrypt`	Isolate connection secrets from code repository; store passwords as irreversible hashes.	GitGuardian identified cleartext secrets in git history; MySQL stores plain-text passwords	No	Yes: Implement `.env` properties reader, purge repository secrets, and integrate `jBCrypt` password hashing.
			Rate Limiting & Asynchronous Job Queues	Throttle excess bursts with HTTP 429 and process calculations via AWS SQS queue.	Unthrottled traffic exhausted thread pool; AWS SQS dependency is infeasible for local lab testing.	No	Yes: Replace AWS SQS cloud queue with an in-process Java executor queue and deploy an embedded Token Bucket filter.
	Reflection:	The audit confirmed that the system operates in an unmitigated state where core application-layer risks are fully exploitable. Crucially, the audit revealed that the IAS 1 proposal for Control 5 relied heavily on external cloud infrastructure (AWS SQS, dedicated EC2 workers), which introduces external financial costs, cloud API dependencies, and complexity that cannot be reliably tested in a standalone academic lab environment. Control 5 must be adapted into an in-process Java concurrency mechanism (such as `ThreadPoolExecutor` paired with Resilience4j/Bucket4j). Furthermore, seed user databases require a clean migration routine to avoid authentication lockout once `bcrypt` is introduced.					
	Updated Controls List & Justifications						
	Controls to Keep (with evidence):	**Control 2 (Parameterized Queries):** Audit demonstrated that input fields directly leak database structures and dump full tables under basic injection tests. Retaining this control provides immediate remediation of RISK-002. **Control 4 (Salted bcrypt Hashing & .env Externalization):** Audit identified plain-text passwords in database tables and administrative connection strings in code files. Retaining this control is essential to comply with RA 10173 and resolve RISK-004. **Control 1 (Server-Side Authorization & JWT Validation):** Audit confirmed horizontal privilege escalation via parameter tampering on `/api/employee/{id}/payroll`. Retaining this control resolves RISK-001.					
	Controls to Revise/Remove (with justification):	**Control 5 (AWS SQS & Cloud Worker Queues):** Revise. The cloud architecture (AWS SQS, external EC2 background worker) introduces external dependencies, billing risks, and network latency unsuited for local lab execution. This is revised to an In-Process Token Bucket Rate Limiting & Java Thread Pool Concurrency Control using local libraries (Resilience4j / Bucket4j and `java.util.concurrent`). **Control 3 (Cryptographic Ledger Hashing):** Defer / Secondary Priority. While valuable for non-repudiation, the primary threats identified in the audit are critical entry-point vulnerabilities (SQLi and credential exposure). Ledger hashing is deprioritized to allow focus on direct perimeter and data protection.					
	New Controls to Add (with justification):	**Database Migration & Legacy Credential Upgrader:** Added as an operational sub-control to Control 4 to automatically detect plain-text passwords upon successful authentication and rehash them dynamically into salted bcrypt records without administrative reset disruption.					